

Strike Sentry — Alpaca AI Trading Agents Hackathon Submission

Team: Harry (solo) — GitHub: harryii-eng

Hackathon: Alpaca AI Trading Agents Hackathon (lablab.ai × Alpaca)

Submission date: August 30, 2026

1. AI Logic

Model: Groq `openai/gpt-oss-20b`, with a deterministic keyword-based fallback if the API is unavailable or returns malformed output — ensures the agent never stalls mid-decision.

Decision flow:

1. Agent receives market context (symbol + signal source: manual / backtest / automation).
2. Groq is prompted to return a strict JSON decision: `{action, confidence, reasoning}`, defaulting to `"hold"` under uncertainty.
3. Decision is logged with a timestamp for reproducibility (`/tmp/agent_orders.log`).

Options strategy: Cash-secured puts. The agent sells out-of-the-money puts (default 8% below the live market price) on liquid single-name

equities, holding
enough cash to cover assignment if exercised.

- **Contract selection:** rather than assuming a strike/expiry exists, the agent queries Alpaca's live options snapshot feed directly and picks the closest strike to its target *among contracts that actually have a real quote* — avoiding the common trap of choosing a theoretically valid but illiquid contract with no tradable price.
 - **Cash requirement:** `strike × 100 × contracts` , minus premium received, checked against real-time buying power before any order is built.
 - **Execution:** `side: sell` , `position_intent: sell_to_open` , submitted as a `limit` order at the live quoted premium via Alpaca's CLI.
 - **Verified live (local/Replit environment):** a real cash-secured put was accepted in paper trading — `AAPL260831P00295000` , strike \$295, premium \$0.18, cash secured \$29,500, order status `accepted` .
-

2. Risk Gates

The agent will not submit an order unless it passes every gate below. This mirrors the guardrails in Alpaca's Paper Trading Skill.

Gate	Enforcement
Paper-only execution	<code>utils/alpaca.js</code> hard-blocks the live endpoint unless <code>ALLOW_LIVE_TRADING=true</code> is explicitly set. Default is always paper.
Buying power check	Every order is preview-checked against <code>GET /v2/account</code> buying power before submission; insufficient funds block the order.
Per-asset-class order validation	<code>validateOrderInputs()</code> enforces Alpaca's order-type / time-in-force / order-class matrix for equities, options, and crypto — rejecting invalid combinations (e.g. crypto <code>stop_limit</code> with <code>ioc</code> , <code>trailing_stop</code> outside <code>day / gtc</code>) before the API call, not after.
qty vs. notional conflict check	Orders cannot mix share/contract quantity and dollar notional.
Preview-before-submit	<code>/order</code> reuses the same payload-building and validation logic as <code>/preview</code> , so nothing reaches submission without having gone through the identical guardrail path.
Full audit log	Every decide/preview/submit/track event is appended to a structured log for post-hoc review.

3. Alpaca Infrastructure Implementation

- **Trading API** — account, market data, and options snapshot endpoints
(`GET /v2/account`, `GET`

`/v2/stocks/{symbol}/trades/latest` ,
`GET /v1beta1/options/snapshots/{symbol}`). Verified live in
production at
`strike-sentry-zk6h.onrender.com/api/agent/positions` .

- **Alpaca CLI** (`alpacacli`) — all order submissions (equity and options)
go through Alpaca's official CLI, shelled out from the Node.js backend via `child_process` , rather than raw REST calls. This satisfies the hackathon's
Trading API + CLI requirement directly: the CLI is the execution layer.
Known deployment limitation: the CLI binary is bundled in `bin/alpaca`
and confirmed working end-to-end in local/Replit testing (see verified order
above), but in the Vercel serverless environment the binary does not resolve
at runtime (`spawn ENOENT`) despite explicit `includeFiles` bundling
configuration and forced executable permissions — a known category of issue
with packaging native binaries into serverless functions. All other
endpoints (account, positions, market data, order preview/validation) are
fully live and verified on Vercel.
 - **Backend:** Node.js / Express, deployed on Vercel.
 - **Frontend:** minimal status page served from `public/` , live at `strike-sentry.vercel.app` .
 - **Order lifecycle tracking:** `alpaca order list --status all`
and
`GET /v2/orders/{id}` for status transitions (accepted, filled, rejected).
-

4. Results (updated through the build window)

- Starting balance: \$100,000
 - Test trades placed during development:
 - Equity market order — AAPL , buy 1 share — **accepted**
 - Cash-secured put — AAPL260831P00295000 , sell 1 contract @ \$0.18 limit, strike \$295 (~8% OTM), expiry 2026-08-31 — **accepted**
 - Ending balance / final P&L: [update once the competition window closes]
-

5. Repo & Demo

- GitHub: <https://github.com/harryii-eng/Strike-sentry>
- Live demo: <https://strike-sentry-zk6h.onrender.com>
- Demo video: [link]